MEMBER A · BACKEND ENGINEER

FastAPI, Agents & *the Spine.*

Your job: make the backend exist, make the agents think, make the WebSocket stream. Everything the frontend shows comes through you.

CLOCK	STACK	RUNTIME	MODELS
24 Hours	Python FastAPI	localhost:8000	Groq (Llama 8B + Qwen3-32B)

DB
Supabase Postgres

PROTOCOLS · SETUP · GATE · CORE AGENTS · WEBSOCKET + DEBATE · INTEGRATION · POLISH · BUG BASH · API MAP

CRITICAL KNOWLEDGE · PROTOCOLS & PATTERNS

What you *must* know before Hour 0.

These protocols govern how your backend talks to the frontend, the LLMs, and the database. Get one wrong and integration at Hour 12 becomes a nightmare.

REST API · HTTP/1.1

FastAPI serves JSON over standard HTTP. All endpoints use `Content-Type: application/json`. Voice endpoints use `multipart/form-data` for audio upload. Always return proper status codes: 200 for success, 422 for validation errors, 429 for rate limits, 500 for server errors. Frontend expects typed JSON responses.

WEBSOCKET · WS://

The debate stream uses `ws://localhost:8000/ws`. Protocol: client sends `{"action":"start","run_id":"..."}`, server pushes events as JSON: `{"stage":"optimist|pessimist|risk|mediator","data":{...}}`. The 2-second stagger

between agent calls spreads TPM load and creates visible sequential arrival on the frontend. If WS fails, frontend falls back to REST polling — your `POST /analyze` must also return the full result synchronously.

CORS · CROSS-ORIGIN

Since frontend runs on `localhost:3000` and backend on `localhost:8000`, CORS is mandatory. Add in `main.py`: `allow_origins=["http://localhost:3000"]`, `allow_methods=["*"]`, `allow_headers=["*"]`. For WebSocket, CORS doesn't apply — WS handshake uses HTTP upgrade. But the initial connection request IS subject to origin checks — verify manually.

CACHING · REDIS / IN-MEMORY DICT

Forecast results are cached with key pattern `forecast:{crop}:{mandi}` and TTL=1h. On localhost, use an in-memory Python dict as primary (skip Upstash Redis entirely — saves setup time). Cache hit means zero Prophet recomputation and zero Groq calls for identical queries. This is what makes demo rehearsals cheap on your 1,000 RPD Qwen budget.

GROQ API · RATE LIMITS

Llama 3.1 8B: 14,400 RPD, 6,000 TPM. Qwen3-32B: 1,000 RPD. The TPM limit is the real bottleneck — bursting 3 parallel agent calls can trip it. Use `asyncio.sleep(2)` stagger between parallel calls. Implement exponential backoff on 429 errors: wait 2s → 4s → 8s. Always wrap Groq calls in `try/except` with the backoff logic in `llm.py`.

SUPABASE · POSTGRES REST

On localhost you connect directly via `supabase-py` client using your project URL + anon key. Schema: `price_records` (commodity, mandi, modal_price, arrival_date, is_anomaly), `forecasts` (crop, mandi, forecast_json, created_at), `sessions` (run_id, status, result_json). The `price_records` table is the foundation — if it's empty, nothing works.

THE #1 LOCALHOST TRAP: PORT CONFLICTS

FastAPI defaults to port 8000. Next.js defaults to port 3000. If you have anything else on those ports, change them immediately. Use `uvicorn main:app --host 0.0.0.0 --port`

`8000 --reload` for dev mode. The `--reload` flag auto-restarts on file changes but can cause WebSocket drops — disable it once WS is stable.

01

HOUR 0-1 · SETUP SPRINT

Boilerplate only. *No features.*

- [ ] **Submit Bhashini registration immediately**

  Go to `bhashini.gov.in` and register for API access. This is async — approval takes up to 24h. Submit and forget. Member C will handle the fallback.

- [ ] **Init FastAPI project structure**

  Create the folder structure: `backend/` with `main.py`, `config.py`, `requirements.txt`. Set up virtual env. Install: `fastapi uvicorn[standard] supabase prophet httpx python-dotenv`. Get `/health` returning JSON immediately.

  `main.py` `config.py` `requirements.txt`

- [ ] **Supabase: create project + run schema.sql**

  Create a free Supabase project. Run the schema SQL to create `price_records`, `forecasts`, `sessions` tables. Copy the project URL and anon key to `.env`. Verify connection with a simple SELECT.

  `db/schema.sql` `db/supabase_client.py`

- [ ] **Set environment variables**

  Create `.env`: `GROQ_API_KEY`, `SUPABASE_URL`, `SUPABASE_KEY`, `ADMIN_TOKEN` (for protected routes). Load with `pydantic-settings` in `config.py`.

  `config.py`

PROPHET PACKAGING — TEST THIS NOW

Prophet has notoriously painful installs. Run `pip install prophet` in your venv before the hackathon starts. If it fails, pre-download the wheel or switch to

> `statsmodels.tsa.holtwinters.ExponentialSmoothing` as the fallback. Do NOT discover this at hour 0.

G1

HOUR 2 · SMOKE TEST GATE

Three checks. *All must pass.*

GATE CRITERIA — DO NOT PROCEED IF ANY FAILS

These are non-negotiable. If you move on with a broken foundation, you'll spend hours 6-12 debugging infrastructure instead of building features.

→ `uvicorn main:app` starts without errors, `/health` returns 200

→ Supabase has 5,000+ rows in `price_records` (depends on Member B's data ingest)

→ `GET /forecast/Tomato/Warangal` returns a 14-day forecast JSON from Prophet

BLOCKED BY MEMBER B — AGMARKNET DATA INGEST MUST COMPLETE FIRST

> **IF PROPHET WON'T INSTALL OR FIT**
>
> Switch to `ExponentialSmoothing` immediately. Don't spend more than 30 minutes debugging Prophet packaging. The forecast doesn't need to be perfect — it needs to exist. You can upgrade to Prophet later if time permits.

02

HOUR 2-6 · CORE AGENT PIPELINE

LLM calls working. *Text-first. No audio.*

- [ ] Build `agents/llm.py` — the Groq wrapper

Single function: `async def call_groq(model, messages, max_tokens)` . Wraps `httpx.AsyncClient` POST to `https://api.groq.com/openai/v1/chat/completions` . Add 429 exponential backoff (2s, 4s, 8s). Add `extract_last_json()` helper to pull JSON from LLM output. This is the foundation every agent uses.

`agents/llm.py`

- [ ] **Build the 3 analyst agents**

`optimist.py` : focuses on upper Prophet band + seasonal uptrend signals. `pessimist.py` : focuses on lower band + glut/oversupply signals. `risk.py` : transport cost, APMC fees (buyer-paid, not farmer), weather shocks. All use `llama-3.1-8b-instant` . Each returns structured JSON with `verdict` (SELL_NOW / HOLD / WAIT), `confidence` (0-100), `reasoning` string, and `data_citations` array.

`agents/optimist.py` `agents/pessimist.py` `agents/risk.py`

- [ ] **Build `agents/advisory.py` — the voice-path advisor**

Single Llama 8B call. Input: forecast summary + profit calc + persona (small/medium/large farmer). Output: Telugu-friendly text advisory. This is the fast path — no debate, no mediator. Voice pipeline uses only this.

`agents/advisory.py`

- [ ] **Build `profit/calculator.py`**

Line-item breakdown: modal_price × quantity, minus transport (hardcoded distance × ₹12/km), minus APMC fee (1.5% paid by buyer — show as effective deduction), minus labor, minus packaging. Return a JSON object with every line item in ₹. The frontend renders this as a receipt-style card.

`profit/calculator.py`

- [ ] **Wire `POST /analyze` route (REST-first)**

Accepts `{crop, mandi, quantity_qtl, persona}` . Calls the 3 agents with `asyncio.gather()` (with 2s stagger), collects results, returns synchronous JSON. WebSocket streaming comes later — but this endpoint must work standalone for Member B to start integrating.

`routers/analyze.py`

HOUR 6 MILESTONE — TEXT-ONLY DEMO WORKS

`POST /analyze` returns 3 agent verdicts + profit breakdown with real data. Frontend can consume this via REST. **Freeze scope here if behind.**

03

HOUR 6-10 · WEBSOCKET + FULL DEBATE

Streaming debate. *Mediator enters.*

- [ ] **Build `agents/debate.py` — deterministic aggregation**

  After 3 agents return verdicts, run 1-round rebuttal: each agent sees the other two verdicts and can revise. Then aggregate: weighted vote (risk agent gets 1.5× weight). Output: `consensus_verdict`, `conflict_score` (HIGH/MEDIUM/LOW), individual revised verdicts.

  `agents/debate.py`

- [ ] **Build `agents/mediator.py` — Qwen3-32B, ONE call**

  The mediator takes the debate output and synthesizes advice for 3 farmer personas (small, medium, large). Uses `qwen-3-32b` on Groq. This is your bottleneck model — 1,000 RPD total. ONE call per query, never more. Cache the result aggressively.

  `agents/mediator.py`

- [ ] **Add WebSocket endpoint `/ws`**

  FastAPI WebSocket: on connect, client sends `{"action":"start","run_id":"..."}`. Server runs the full pipeline and pushes events as each stage completes: `{"stage":"optimist","data":{...}}`, then pessimist, risk, debate, mediator. The 2s stagger becomes visible sequential arrival on the frontend — dramatic pacing, not latency.

  `routers/analyze.py`

- [ ] **Wire voice endpoints (basic)**

  `POST /voice/transcribe`: accept audio file, call Groq Whisper (`whisper-large-v3`, `language="te"`), return `{text_en, text_te}`. `POST /voice/query`: full voice loop — transcribe → translate → advisory → TTS. The TTS part depends on Member C's Bhashini/AI4Bharat work.

  `routers/voice.py` `voice/stt.py` ⚡ TTS INTEGRATION DEPENDS ON MEMBER C

WEBSOCKET-EVENT-FORMAT

```
// WebSocket event format — Member B needs this contract { "stage": "optimist" | "pessimist" | "risk" | "debate" | "mediator", "data": { "verdict": "SELL_NOW" | "HOLD" | "WAIT", "confidence": 85, "reasoning": "...", "data_citations": ["7-day delta: +12%", "Prophet upper: ₹2,340"] }, "timestamp": "2025-01-15T10:30:00Z" }
```

04

HOUR 10-14 · INTEGRATION · EXPECT PAIN

Backend meets Frontend. *CORS, types, formats.*

- [ ] **Pair with Member B for 3 uninterrupted hours**

  This is where CORS mismatches, JSON format differences, and type errors surface. Real API meets real frontend. Fix issues live, together. Do NOT try to fix integration bugs over Slack messages.

  ⚡ PAIRED WORK WITH MEMBER B — CLEAR YOUR SCHEDULE

- [ ] **Verify CORS is correctly configured**

  Common bug: CORS middleware must be added BEFORE route registration. If you see `Access-Control-Allow-Origin` errors in the browser console, check middleware order. For localhost, allow origin `http://localhost:3000` (not `https`, not without port).

- [ ] **Build `fallback/canned_demo.py`**

  Pre-compute 3 full responses (Tomato/Warangal, Onion/Hyderabad, Chili/Karimnagar). Store as static JSON. Serve via `GET /demo/canned/{crop}/{mandi}`. This is your WiFi-dies insurance and also the `?demo=canned` flag for frontend. Zero external API calls.

  `fallback/canned_demo.py`

HOUR 14 · FULL DRY-RUN GATE

Walk the demo end-to-end. Text path + voice path. Will have bugs. Fix only show-stoppers. **This is the milestone that decides whether you win or place.**

→ POST /analyze returns all 4 agent results via WebSocket within 15s

→ Voice query returns Telugu text within 7s

→ /demo/canned works with zero external calls

→ Frontend displays agent cards, price chart, profit breakdown with real data

---

05

HOUR 14-18 · SECOND-TIER + POLISH

Weather, SMS, FPO support. *Only if core is solid.*

- [ ] **Weather integration (`data/weather.py`)**

  Open-Meteo API — no key required. Fetch rainfall data for mandi locations. Optionally add as Prophet regressor for Tomato/Warangal. Only enable if measurable MAPE improvement — verify before expanding to other crops.

  `data/weather.py`

- [ ] **SMS alerts (`alerts/sms.py`)**

  Twilio trial account. Send Unicode Telugu SMS to pre-verified numbers only. `POST /alerts/sms/subscribe` and `POST /alerts/sms/send` . Seed a "3 days ago" alert for demo narrative. **Critical: Twilio trial can only send to pre-verified numbers. Pre-verify your teammate's phone.**

  `alerts/sms.py` `routers/alerts.py`

- [ ] **FPO choropleth endpoint**

  `GET /fpo/choropleth/{crop}` — returns GeoJSON FeatureCollection with district-level price stats. Member B's frontend will consume this for the map. Cut entirely at Hour 16 if behind schedule.

  `routers/fpo.py`

- [ ] **In-memory caching across all endpoints**

  Add dict-based cache to forecast, analyze, and profit endpoints. Key: `f"{crop}:{mandi}:{quantity}"` . TTL: 1 hour (use timestamp check). This prevents burning Groq rate limits during demo rehearsals.

06

HOUR 18-22 · BUG BASH + HARDENING

Graceful degradation. *Never show "error".*

- [ ] **Add graceful degradation everywhere**

  If Prophet fails → fall back to 7-day rolling average. If Groq 429 → return cached result or canned response. If Supabase down → serve from local cache. The UI should NEVER show a raw error trace. Every failure path returns something useful.

- [ ] **Run the verification checklist**

  All 8 steps from the main document. At Hour 18, run them once. At Hour 21, run them again. Any failure is still fixable at these windows; a failure during the actual demo is not.

- [ ] **Support Member C's demo rehearsals**

  Keep the backend running. Monitor for 429 errors during rehearsal. If rate limits hit, switch to canned mode. Be available for quick fixes — "the profit calculator returns negative" type bugs.

  ⚡ MEMBER C NEEDS STABLE BACKEND FOR REHEARSAL

> **HOUR 22-24: PITCH SUPPORT + SLEEP**
>
> Help Member C finalize the demo flow. Verify `?demo=canned` works without any network calls. Record a backup video of the demo. Then sleep. Minimum 2 hours. Present fresh — more hackathons are lost to exhaustion in Q&A than to technical failure.

QUICK REFERENCE · YOUR API SURFACE

Every endpoint *you own.*

GET
`/health`
Service status · Prophet ready · cache stats

WS
`/ws`
Streams agent results as each stage completes

POST
`/analyze`
Full debate pipeline · returns run_id + sync result

GET
`/forecast/{crop}/{mandi}`
14-day Prophet forecast + rolling MAPE

POST
`/profit/calculate`
Line-item ₹ breakdown

POST
`/voice/transcribe`
Audio → Telugu/English text

POST
`/voice/query`
Full voice loop — STT → LLM → TTS

POST
`/alerts/sms/subscribe`
Register phone for price alerts

POST
`/alerts/sms/send`
Trigger SMS notification

GET

`/fpo/choropleth/{crop}`

GeoJSON district-level price stats

POST

`/admin/ingest`

Token-gated · CSV ingest trigger

GET

`/demo/canned/{crop}/{mandi}`

Offline pre-computed response